Estudi Previ Pràctica 1

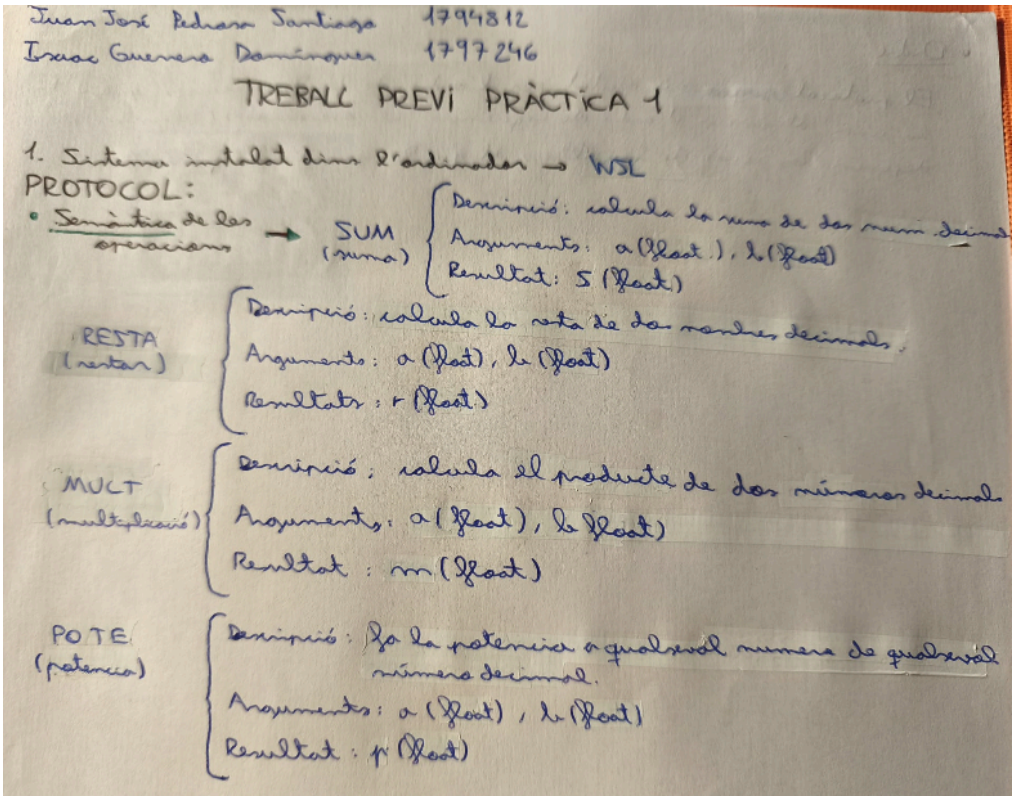
Alumne 1: Juanjo Pedrosa Santiago NIU: 1794812
Alumne 2: Isaac Guerrero Domínguez NIU: 1797246

Objectiu de la Pràctica: Conectar 2 clients a un únic servidor en WSL (UBUNTU) i comunicar-se entre ells. També observarem que les dades dels processos realitzats client a client queden registrades al servidor.

PRÈVIA INICIAL

El nostre projecte és una calculadora que permet fer Suma, Resta, Multiplicació i Potències de qualsevol nombre decimal.

Protocol (imatge):



PROTOCOL:

- **Semàntica de les operacions:**
  - **SUM** (suma)
    - **Descripció:** calcula la suma de dos números decimals.
    - **Arguments:** a (float) , b (float)
    - **Resultat:** s (float)
  - **RESTA** (restar)
    - **Descripció:** calcula la resta de dos nombres decimals
    - **Arguments:** a (float) , b (float)

        - **Resultat:** r (float)
    - **MULT** (multiplicació)
        - **Descripció:** calcula el producte de dos números decimals.
        - **Arguments:** a (float) , b (float)
        - **Resultat:** m (float)
    - **POTE** (potència)
        - **Descripció:** fa la potència a qualsevol número de qualsevol número decimal.
        - **Arguments:** a (float) , b (float)
        - **Resultat:** p (float)

Sintaxis (funcions matemàtiques):

Les funcions matemàtiques que engloben el nostre projecte (que es troben al *client*):

```
if (opcio == 2) sprintf(peticion, "SUMA %.2f %.2f", num1, num2);
else if (opcio == 3) sprintf(peticion, "RESTA %.2f %.2f", num1, num2);
else if (opcio == 4) sprintf(peticion, "MULT %.2f %.2f", num1, num2);
else if (opcio == 5) sprintf(peticion, "POTE %.2f %.2f", num1, num2);
```

Ordre:

El protocol no es veu afectat ni influenciat per l'ordre de les funcions, ja que hi ha una absència d'ordre de seqüència. Les operacions/funcions no depenen les unes de les altres, pel que l'ordre no té cap influència dins del protocol.

SIMULACIÓ PRÀCTICA

Servidor en funcionament esperant la connexió d'un client:

```
jotavait@Sonic:~/practica1$ ./server 54812
Servidor en funcionament, esperant connexions de client...
```

Menú d'interacció client:

```
jotavait@Sonic:~/practica1$ ./client 127.0.0.1 54812
Connectat al servidor 127.0.0.1:54812

================ MENU CLIENT ================
1. Enviar missatge simple
2. Sumar dos nombres (SUMA)
3. Restar dos nombres (RESTA)
4. Multiplicar dos nombres (MULT)
5. Potencia (POTE)
6. Sortir (EXIT)
Selecciona una opció:
```

Exemple 1 Missatge entre clients:

```
Selecciona una opció: 1
Introdueix el missatge: hola mundo
Resposta del servidor: hola mundo. Missatge rebut
```

```
Missatge rebut del client (PID: 615): MSG hola mundo
```

Exemple 2 Multiplicació:

```
Selecciona una opció: 4
Introdueix el primer número: 2
Introdueix el segon número: 2
Resposta del servidor: Resultat: 4.00
```

```
Missatge rebut del client (PID: 615): MULT 2.00 2.00
```

El **PID** ***(Process ID)*** és un nombre enter únic que el sistema operatiu (com Linux, Unix o Windows) assigna a cada programa o procés que es troba en execució.

FUNCIONS:

```c
void procesar_comando(char *peticion, char *respuesta) {
    char cmd[30]; // ---------> 10 caracters per almecenar els nombres
    double a, b, res;

    if (sscanf(peticion, "%s %lf %lf", cmd, &a, &b) == 3) {  //lf variab
        if (strcmp(cmd, "SUMA") == 0) {
            sprintf(respuesta, "Resultat: %.2f\n", a + b);
        } else if (strcmp(cmd, "RESTA") == 0) {
            sprintf(respuesta, "Resultat: %.2f\n", a - b);
        } else if (strcmp(cmd, "MULT") == 0) {
            sprintf(respuesta, "Resultat: %.2f\n", a * b);
        } else if (strcmp(cmd, "POTE") == 0) {
            sprintf(respuesta, "Resultat: %.2f\n", pow(a, b));   // Pow
        } else {
            strcpy(respuesta, "ERROR: Comanda no reconeguda.\n");
        }
    } else {
        strcpy(respuesta, "ERROR: Format de instruccio invalid.\n");
    }
}
```

SOCKETS:

Durant la pràctica hem observat que per realitzar la connexió cal fer ús de ***sockets***:

```
if ((sock = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
    perror("Error en crear el socket");  // perror
}
```

Un ***socket*** funciona com un punt final d'un canal de comunicació bidireccional entre dos programes (ja sigui al mateix ordinador oa través d'una xarxa com a Internet).

htonl: Converteix num de 32 bits(4 Bytes) de l'ordre de l'equip a l'ordre de la red.
htons: Converteix num de 16 bits(2 Bytes) de l'ordre de l'equip a l'ordre de la red.

BIND:

```
if (bind(sock, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("Error en fer el bind");
    close(sock);
    exit(EXIT_FAILURE);
}
```

La funció ***bind ()*** és una funció del sistema operatiu que associa un socket de xarxa amb una adreça IP i un port específics.

LISTEN:

```
if (listen(sock, 3) < 0) {
    perror("Error en fer el listen");
    close(sock);
    exit(EXIT_FAILURE);   // ---------
}
```

La funció ***listen ()*** posa el socket del servidor en un estat passiu d'escolta per acceptar sol·licituds o intents de connexió que provenen dels diferents clients.

Que fa el Fork?.

La funció ***fork ()*** una crida al sistema (system call) que crea un nou procés (procés fill) duplicant exactament el procés actual (procés pare). Quan es combina amb sockets, la funció ***fork()*** s'utilitza principalment per construir servidors concurrents capaços d'atendre múltiples clients alhora.

Resum general funcionament d'un servidor:

- **Creació i escolta:** El procés servidor principal crea un socket (socket()), l'assigna a un port (bind()) i es queda esperant connexions entrants (listen() i accept()).

- **Arribada d'un client:** Quan un client es connecta, la funció accept() desbloqueja i torna un nou descriptor del socket dedicat exclusivament a la comunicació amb aquest client específic, mentre el sòcol original segueix lliure per escoltar noves connexions.

- **La trucada a fork():** En aquest moment, el servidor executa fork(). El sistema operatiu duplica el procés: El Procés Fill: Hereta una còpia del sòcol connectat amb el client. El fill s'encarrega d'atendre completament el client (llegir les seves peticions, processar-les i enviar-li respostes). Quan acaba la comunicació, el fill tanca el seu sòcol i finalitza. El Procés Pare: Torna immediatament al bucle principal per tornar a trucar a accept() i esperar el següent client que intenti connectar-se.

Client:

La funció principal del client en una aplicació de xarxa és actuar com iniciador i consumidor del servei.

- **Crear el vostre propi socket**: Configura un punt d'entrada a la màquina per poder comunicar-se per la xarxa.

- **Connectar-se al servidor:** Truqueu a la funció de connexió (connect()) especificant l'adreça IP i el port on el servidor està escoltant.

- **Enviar dades (Petició):** Envia informació útil al servidor (per exemple, una consulta, un fitxer, una ordre o una petició HTTP).

- **Rebre dades (Resposta) i tancar:** Es queda esperant la resposta del servidor, la processa (la mostra a la pantalla, la guarda, etc.) i finalment tanca la connexió (close()).

En resum: El client és el que fa el primer pas per demanar alguna cosa, espera que el servidor li el torni, i tanca la connexió en acabar.